Classes and Objects

Complete Notes, Python Solutions & Design Patterns

Topic Overview

Core Concepts

- **Class:** A blueprint or template that defines properties (data) and member functions (behavior).
- **Object:** An instance of a class created using the class constructor.
- **Member functions:** Functions defined inside a class that operate on the object's own data (`self` in Python, `this` in Java/C++).
- **Constructor:** A special method (`__init__` in Python) that runs automatically when an object is created, used to initialize properties.

Python Syntax Reference

```
class ClassName:
    def __init__(self, prop1, prop2):
        self.prop1 = prop1
        self.prop2 = prop2

    def memberFunction(self):
        # operate on self.prop1, self.prop2 ...
```

Python Tasks and Solutions

Task 1: Playlist Class – Properties & Instantiation

```
from datetime import date

class Playlist:
    def __init__(self, name, createdOn, isPublic):
        self.name = name
        self.createdOn = createdOn
        self.isPublic = isPublic

# --- Instantiate an object and print all properties ---
p1 = Playlist("Morning Vibes", date(2025, 1, 15), True)

print("Name:      ", p1.name)
print("Created On:", p1.createdOn)
print("Is Public: ", p1.isPublic)
```

Task 2: togglePublic() Member Function

```
from datetime import date

class Playlist:
    def __init__(self, name, createdOn, isPublic):
        self.name = name
        self.createdOn = createdOn
        self.isPublic = isPublic

    def togglePublic(self):
        """Switches isPublic between True and False each call."""
        self.isPublic = not self.isPublic

# --- Demonstrate toggling twice ---
p1 = Playlist("Morning Vibes", date(2025, 1, 15), True)
print("Before toggle:", p1.isPublic)   # True

p1.togglePublic()
print("After toggle 1:", p1.isPublic)  # False

p1.togglePublic()
print("After toggle 2:", p1.isPublic)  # True (back to original)
```

Task 3: FoodOrder Class with markDelivered()

```
class FoodOrder:
    def __init__(self, orderId, restaurantName, isDelivered=False):
        self.orderId = orderId
        self.restaurantName = restaurantName
        self.isDelivered = isDelivered

    def markDelivered(self):
        self.isDelivered = True
        print(f"Order #{self.orderId} from {self.restaurantName} has been
            delivered!")

# --- Instantiate and call markDelivered() ---
order1 = FoodOrder(101, "Pizza Palace")
print("Before:", order1.isDelivered)   # False
order1.markDelivered()
print("After: ", order1.isDelivered)   # True
```

Task 4: Extended Playlist – addSong() with songs Array

```
from datetime import date

class Playlist:
    def __init__(self, name, createdOn, isPublic):
        self.name = name
        self.createdOn = createdOn
        self.isPublic = isPublic
        self.songs = []                      # empty array in constructor
```

```
    def togglePublic(self):
        self.isPublic = not self.isPublic

    def addSong(self, songTitle):
        self.songs.append(songTitle)

    def showSongs(self):
        print(f"\nSongs in '{self.name}':")
        for i, song in enumerate(self.songs, start=1):
            print(f"{i}. {song}")

p1 = Playlist("Morning Vibes", date(2025, 1, 15), True)
p1.addSong("Sunrise Avenue")
p1.addSong("Chill Waves")
p1.addSong("Morning Coffee")
p1.showSongs()
```

Task 5: Refactored FoodOrder – Object as Constructor Parameter

```
class FoodOrder:
    def __init__(self, order):           # takes ONE object/dict parameter
        self.orderId = order["orderId"]
        self.restaurantName = order["restaurantName"]
        self.isDelivered = order["isDelivered"]

    def markDelivered(self):
        self.isDelivered = True
        print(f"Order #{self.orderId} from {self.restaurantName} delivered!
            ")

    def display(self):
        print(f"Order #{self.orderId} | {self.restaurantName} | Delivered:
            {self.isDelivered}")

order_data = {
    "orderId": 102,
    "restaurantName": "Sushi World",
    "isDelivered": False
}

order1 = FoodOrder(order_data)
order1.display()
order1.markDelivered()
order1.display()
```

Summary & Key Takeaways

#	Concept	Description
1	**State Isolation**	Each object maintains its own copy of attributes, preventing crosstalk or global corruption bugs.
2	**Constructor Initialization**	Initializing collections (like `self.songs = []`) inside `__init__` avoids shared reference bugs across instances.
3	**Object Parameters**	Passing a dictionary/object to constructors simplifies signatures and integrates cleanly with external JSON APIs.

Key Takeaway: A class bundles *state* (properties) and *behavior* (member functions) into a unified module. Objects encapsulate their own data, guaranteeing consistency and safety across complex applications.